Frameworks Added — Attack & Pentest Framework

Cerberus backend · `app/ai_engine/pentest/`

commits ac4dd5f + b83606f + 8fd3935 · 31 files · ~2,800 lines

What was added

A complete **attack / pentest framework** inside the backend's AI engine, built tool-agnostic from the ground up. It runs bounded, safety-gated attack modules against in-scope targets, streams findings as they are produced, and can drive external security tools (Metasploit today; nmap / sqlmap / nuclei later) through a uniform adapter layer.

```
app/ai_engine/pentest/
├── core/          AttackModule ABC · PentestContext · ScopeConfig · Finding
│                  Evidence · Severity/Impact/Confidence enums · SafetyGuard
│                  · exceptions · JSON attack datasets
├── engine/        ExecutionEngine (fail-soft) · ModuleRegistry (auto-discovery)
│                  · ExecutionScheduler (sequential, streaming, cancellable)
├── modules/       web/ (3) · network/ (2) · auth/ (1) · tools/ (1)
├── integration/   engagement adapter → Vulnerability / VulnerabilityEvidence
└── tools/         ToolAdapter ABC · ToolRegistry · Metasploit adapter
```

Module catalog (7 modules, 4 categories)

Module	Category	MITRE	Impact	Severity	What it does
`sqli_probe`	web	T1505	proof-of-concept	HIGH	Error-based and boolean-based SQL injection detection
`xss_probe`	web	T1505	proof-of-concept	HIGH	Reflected cross-site scripting detection (canary payload)
`path_traversal_probe`	web	T1505	proof-of-concept	HIGH	Local/remote file inclusion (path traversal) detection
`port_scanner`	network	T1046	proof-of-concept	INFO	Bounded TCP connect port discovery (per-port evidence)
`service_fingerprint`	network	T1592	proof-of-concept	INFO	HTTP technology fingerprinting via response signatures
`credential_tester`	auth	T1078	proof-of-concept	HIGH	Single-shot credential validation — no brute force, no password spraying
`metasploit_scanner`	tools	T1046, T1087	proof-of-concept	INFO	Runs a Metasploit auxiliary module via msfrpcd (see the two Metasploit PDFs in this folder)

Engine & scheduling

- **ExecutionEngine** — runs a module against a context, returns an `ExecutionReport`; a failing module yields an ERROR finding instead of crashing the engagement (fail-soft).
- **ModuleRegistry** — package auto-discovery: dropping a module file into `modules/<category>/` registers it.
- **ExecutionScheduler** — sequential multi-module runs with `on_finding` streaming callbacks and cooperative cancellation.

Safety model (code-enforced, not just documented)

1. **Scope gate** — `SafetyGuard` checks the target against the engagement's allowed targets/ports and raises `OutOfScopeError` *before* any module code runs.
2. **Rules-of-Engagement gate** — every module declares an `impact` level; the guard raises `RoEViolationError` if it exceeds the engagement's `max_impact` ceiling.
3. **Proof-of-concept by default** — all built-in modules are bounded and non-destructive; the credential tester performs exactly one validation attempt (no brute force).

4. **Opt-in external tools** — adapters ship with `enabled_by_default = False` and fail soft when their daemon is unreachable.

Results & integration

- **Finding** — module, vulnerability type, severity, title, description, target, confidence, MITRE technique, evidence list, verified / false-positive flags.
- **Engagement adapter** — builds a `PentestContext` from an engagement record, maps findings to `Vulnerability` + `VulnerabilityEvidence` rows (INFO/ERROR findings excluded by default), and persists them.
- **Datasets** — JSON payload datasets (web payloads, traversal paths) loaded from the package's `datasets/` directory.

File inventory (31 files)

Layer	File	Lines	Purpose
core	`core/base_module.py`	132	`AttackModule` ABC — metadata + async `run()` yielding findings
	`core/context.py`	213	`PentestContext`, `ScopeConfig`, `ImpactLevel`
	`core/results.py`	198	`Finding`, `Evidence`, severity/confidence enums
	`core/safety.py`	84	`SafetyGuard` — scope + RoE impact gates
	`core/exceptions.py`	35	`PentestError` hierarchy incl. `ToolUnavailableError`
	`core/datasets.py`	69	JSON attack-dataset loading
engine	`engine/execution_engine.py`	173	`ExecutionEngine` + `ExecutionReport` (fail-soft)
	`engine/module_registry.py`	149	`ModuleRegistry` with package auto-discovery
	`engine/scheduler.py`	150	`ExecutionScheduler` — sequential, streaming, cancellable
web modules	`modules/web/sqli_probe.py`	248	SQL injection detection (error-based + boolean-based)
	`modules/web/xss_probe.py`	172	Reflected XSS detection
	`modules/web/path_traversal.py`	160	Path-traversal / file-inclusion detection
network modules	`modules/network/port_scanner.py`	152	Bounded TCP connect port discovery
	`modules/network/service_fingerprint.py`	122	HTTP technology fingerprinting
auth module	`modules/auth/credential_tester.py`	197	Single-shot credential validation
tools layer	`tools/base_tool.py`	74	`ToolAdapter` ABC + `ToolStatus`
	`tools/metasploit_adapter.py`	214	msfrpcd XML-RPC adapter (stdlib only)
	`tools/tool_registry.py`	98	`ToolRegistry` with auto-discovery
	`modules/tools/metasploit_scanner.py`	239	Metasploit auxiliary-module runner (T1046/T1087)
integration	`integration/engagement_adapter.py`	199	Engagement ↔ context mapping; findings → Vulnerability persistence

Plus `__init__.py` package files re-exporting the public API, and the shared web-module helpers in `modules/web/common.py` (17 lines).

Verification

- **76/76 unit tests** (61 framework + 15 tool-integration, including a real in-process fake msfrpcd).
- ruff / black / mypy clean across the package.
- Live smoke: SQLi HIGH verified with evidence, port scan, fingerprint, out-of-scope target refused.
- API healthy; framework connected as a first-class backend package (endpoint / AI-agent wiring is the next phase).